



Управление данными и искусственный интеллект
Лекция по дисциплине:
“Глубокое обучение в компьютерном зрении”



Российский университет
дружбы народов

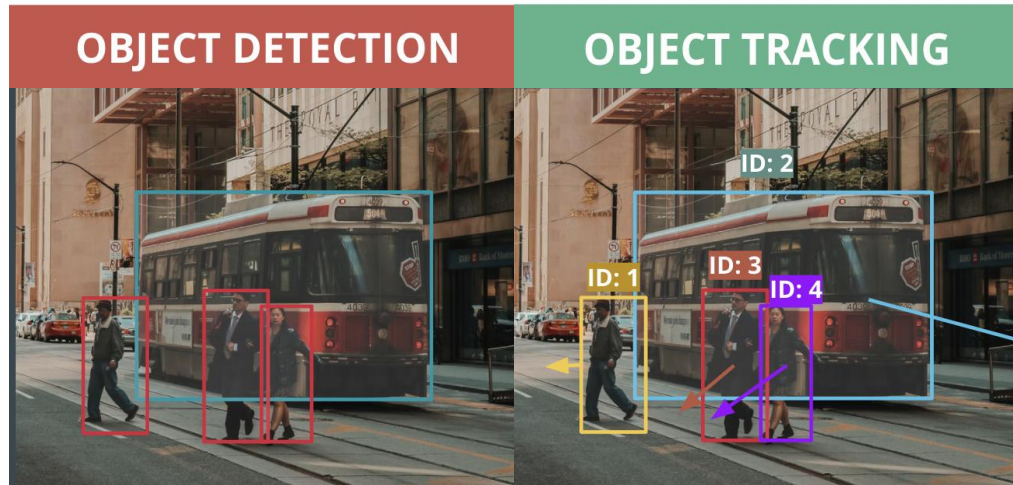
ВИЗУАЛЬНОЕ ОТСЛЕЖИВАНИЕ ДВИЖУЩИХСЯ ОБЪЕКТОВ

Преподаватели: Перчихин Олег Игоревич
Федоренко Анастасия Олеговна

ОПРЕДЕЛЕНИЕ ЗАДАЧИ ОТСЛЕЖИВАНИЯ

Задача общего визуального отслеживания выполняется в домене изображений без предварительных знаний о классах объектов. Отслеживание формулируется в 2D-пространстве и касается одного объекта (цели), который должен быть найден на всех кадрах последовательности, несмотря на наличие других движущихся объектов (дистракторов) или частичные перекрытия.

В отличие от задачи обнаружения объектов, цель отслеживания заключается не только в локализации объекта, но и в поддержании его идентичности во времени, что критически важно для реальных приложений, включая беспилотные автомобили, системы видеонаблюдения, робототехнику и другие динамические системы, требующие понимания временной эволюции сцены



ОБНАРУЖЕНИЕ ОБЪЕКТОВ VS ОТСЛЕЖИВАНИЕ ОБЪЕКТОВ

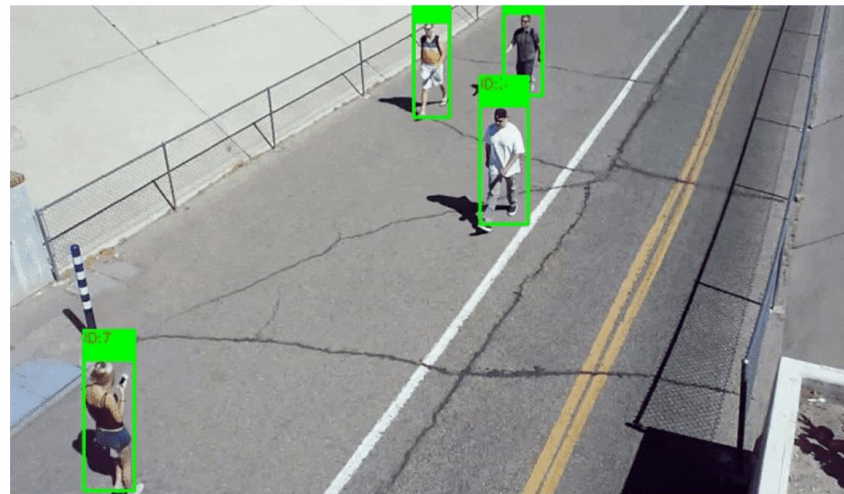
В классических курсах компьютерного зрения часто внимание сосредотачивается на обнаружении объектов (object detection), то есть на определении координат ограничивающих рамок (bounding boxes) и классов объектов в отдельных изображениях. Однако для практических систем, особенно динамических, таких как автономные транспортные средства, этого недостаточно.

Недостатки обнаружения:

- Отсутствие временной информации: детектор не учитывает движение объекта между кадрами.
- Нет идентификации: на каждом кадре объект рассматривается как новый.
- Невозможно оценить скорость, направление или продолжительность нахождения объекта в сцене.

Отслеживание объектов (object tracking) решает эти проблемы, добавляя к детекции временную компоненту и обеспечивая:

- сохранение идентификатора объекта во времени,
- возможность вычисления скорости, направления и траектории,
- оценку долгосрочной видимости и состояния (например, частичное перекрытие, исчезновение).



ОСНОВНЫЕ ЭТАПЫ ОТСЛЕЖИВАНИЯ ОБЪЕКТОВ

Процесс отслеживания объектов в видео представляет собой сложную многоэтапную процедуру, требующую интеграции различных методов компьютерного зрения, машинного обучения и теории оценивания. Каждый этап решает специфическую подзадачу и вносит критический вклад в общую точность и устойчивость системы.

1

Обнаружение объектов

На первом этапе используются современные детекторы для получения ограничивающих рамок на каждом кадре видеопоследовательности.

2

Ассоциация данных

Необходимо сопоставить объекты, найденные на разных кадрах, чтобы определить, какой объект на кадре t соответствует объекту на кадре $t-1$.

Методы сопоставления:

- **Двудольное сопоставление** — построение графа связей между объектами
- **Венгерский алгоритм** — оптимальное решение задачи назначения

3

Фильтрация и предсказание движения

Применение методов, моделирующих динамику объектов для предсказания их будущего положения и компенсации ошибок детекции.

Основные подходы:

- **Фильтр Калмана** — линейная аппроксимация движения с гауссовским шумом
- **Фильтр частиц** — нелинейные и стохастические модели движения
- **Глубокие нейронные сети** — DeepSORT, CenterTrack, ByteTrack, объединяющие визуальные признаки и динамику



Формальное определение задачи отслеживания

Пусть дана видеопоследовательность изображений и ограничивающая рамка в первом кадре, описываемая четырьмя параметрами:

$$(x_0, y_0, w_0, h_0)$$

где x_0, y_0 — координаты верхнего левого угла, а w_0, h_0 — ширина и высота рамки соответственно.

Необходимо предсказать положение объекта на всех последующих кадрах:

$$(x_k, y_k, w_k, h_k), \quad k > 0$$

при условии, что при вычислении рамки для кадра K могут использоваться только данные предыдущих кадров $k \leq K$.

Таким образом, задача является **онлайн-задачей прогнозирования положения и идентичности объекта**, что накладывает дополнительные требования на вычислительную эффективность алгоритмов. В отличие от офлайн-методов, которые могут использовать информацию из будущих кадров для уточнения траекторий, онлайн-алгоритмы должны принимать решения немедленно, основываясь только на доступной на текущий момент информации.



Ключевые ограничения

- Онлайн-обработка данных
- Каузальность: использование только прошлых кадров
- Реальное время выполнения
- Устойчивость к перекрытиям

ОСНОВНЫЕ ЭЛЕМЕНТЫ ОТСЛЕЖИВАНИЯ ОБЪЕКТОВ: ОТСЛЕЖИВАНИЕ ОТ КАДРА К КАДРУ

Первостепенная цель алгоритмов отслеживания объектов заключается в **сопоставлении ограничивающих рамок** (bounding boxes) между последовательными кадрами видеопоследовательности. Это позволяет определить, какой объект в кадре t соответствует объекту в кадре $t+1$, сохранив при этом его идентичность.

Для наглядности рассмотрим **пример сцены, снятой с камеры беспилотного автомобиля**. Цель — отслеживать движение транспортных средств в условиях плотного трафика, изменяющегося освещения и частичных перекрытий.

На исходном изображении присутствует статический фон (дорожная разметка, здания, деревья) и движущиеся объекты (автомобили, велосипедисты, пешеходы).



Исходная сцена

ОСНОВНЫЕ ЭЛЕМЕНТЫ ОТСЛЕЖИВАНИЯ ОБЪЕКТОВ: ОТСЛЕЖИВАНИЕ ОТ КАДРА К КАДРУ

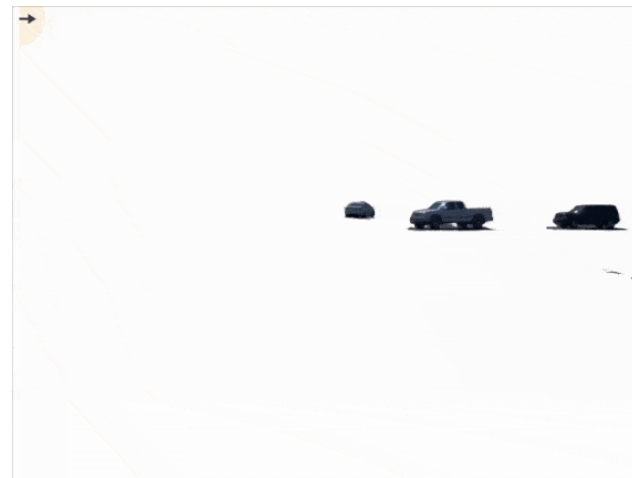
Если фон удалить, то останутся только **динамические элементы сцены** — именно они и являются объектами отслеживания.

Задача идентификации

В процессе анализа подобных видеопоследовательностей возникает задача **корректного присвоения идентификаторов** каждому объекту и последующего отслеживания этих идентификаторов от кадра к кадру.

Особую сложность представляют ситуации, когда объекты:

- Частично перекрывают друг друга
- Временно покидают поле зрения
- Имеют схожий внешний вид



- ❑ Надежное отслеживание требует комбинации нескольких подходов: детекции объектов, анализа их визуальных признаков, моделирования движения и интеллектуального сопоставления между кадрами. Каждый из этих компонентов будет подробно рассмотрен далее.

ОСНОВНЫЕ ЭЛЕМЕНТЫ ОТСЛЕЖИВАНИЯ ОБЪЕКТОВ: ОТСЛЕЖИВАНИЕ ОТ КАДРА К КАДРУ

Для выполнения отслеживания объектов в видеопоследовательности необходимо в первую очередь **извлечь отдельные кадры** из видеопотока. Это позволяет анализировать динамику движения объектов во времени и сопоставлять их между последовательными изображениями.

После извлечения видеок кадров задача заключается в **присвоении уникальных идентификаторов (ID)** каждому обнаруженному объекту, чтобы сохранить их непрерывную идентичность во времени.

Рассмотрим пример с последовательными кадрами — например, третьим и четвёртым. Цель состоит в том, чтобы определить, какой объект на кадре 4 соответствует каждому из объектов, найденных на кадре 3, и тем самым обеспечить корректное продолжение их треков.



1. ОБНАРУЖЕНИЕ ОБЪЕКТОВ (OBJECT DETECTION)

На первом этапе производится детекция объектов на каждом кадре независимо от других кадров. Для этого применяются нейросетевые модели, например, YOLO, Faster R-CNN, SSD и др., которые обрабатывают изображение и выдает координаты ограничивающих рамок вместе с классами объектов и степенью уверенности в предсказании.

Результат детектора:

- Координаты ограничивающих рамок (x, y, w, h)
- Класс объекта (автомобиль, пешеход, велосипед и т.д.)
- Степень уверенности (confidence score)



Проблема идентичности

Детектор не знает, что объект на кадре t — это тот же объект, что был на кадре $t-1$. Требуется дополнительный механизм для связывания обнаружений между кадрами.

Следовательно, требуется следующий шаг — **Ассоциация (объединение) данных между кадрами**, то есть установление соответствия между объектами, обнаруженными на последовательных кадрах видео.

2. АССОЦИАЦИЯ ДАННЫХ

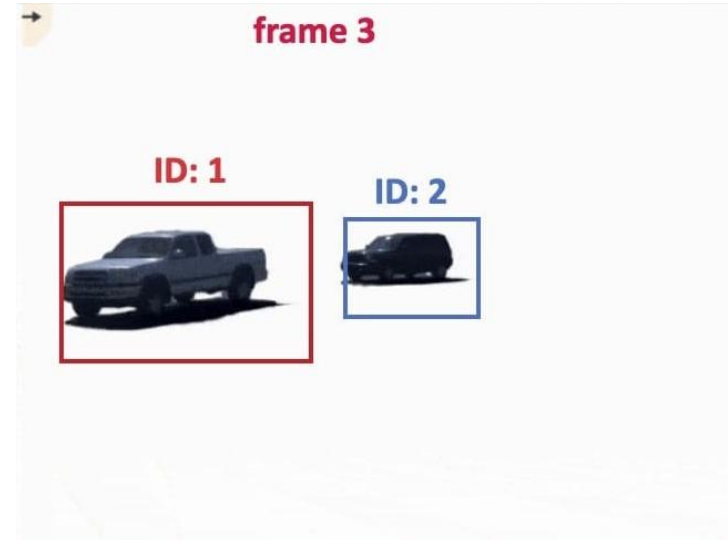
Цель данного этапа — присвоить каждому обнаруженному объекту **уникальный идентификатор (ID)** и обеспечить его корректное соответствие между последовательными кадрами. В контексте многообъектного отслеживания (MOT, Multi-Object Tracking) это означает, что каждый объект должен сохранять свой ID на протяжении всего видео, даже если он временно исчезает из поля зрения или частично перекрывается другими объектами.



АССОЦИАЦИЯ ДАННЫХ

Итак, на первом этапе **каждому объекту в кадре $t-1$** присваивается **уникальный идентификатор (ID)**, который будет использоваться для отслеживания его в последующих кадрах.

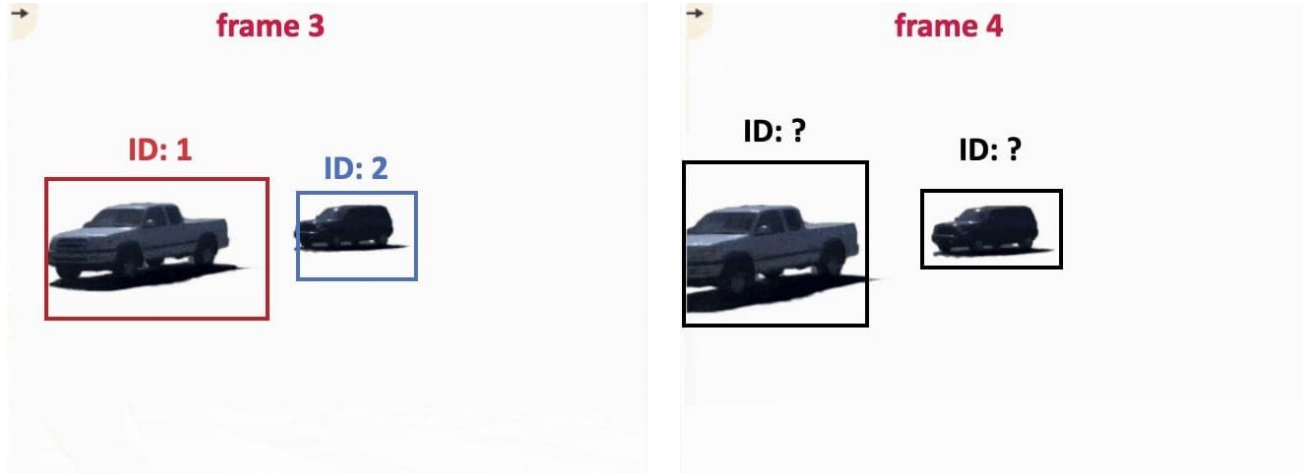
Идентификатор для каждого объекта можно присвоить совершенно случайным образом



АССОЦИАЦИЯ ДАННЫХ

Далее необходимо **выполнить сопоставление объектов между кадрами $t-1$ и t** .
Цель — определить, какой объект в текущем кадре соответствует объекту из предыдущего.

Возникает вопрос: **каким образом выполняется это сопоставление?**



АССОЦИАЦИЯ ДАННЫХ

Это выполняется следующим образом: запускается **процесс ассоциации данных**, в котором для каждого объекта из кадра $t-1$ подбирается наиболее вероятное соответствие в кадре t .

Для этого используются несколько **критериев сопоставления**, включая:

Геометрические метрики

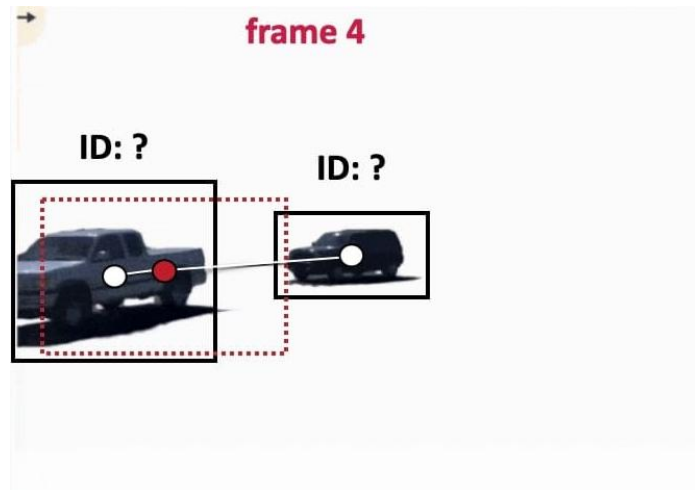
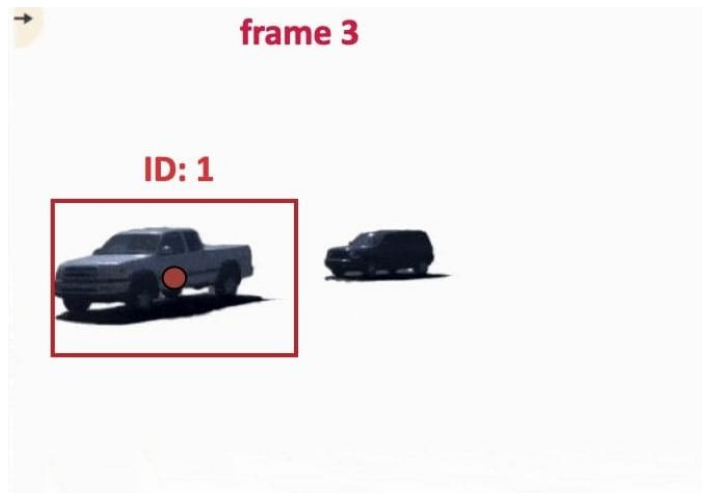
- Евклидово расстояние между центрами рамок
- Изменение размеров и пропорций рамки
- IoU (Intersection over Union) — пересечение рамок

Семантические метрики

- Класс объекта (автомобиль $\neq$ велосипедист)
- Косинусное расстояние в пространстве признаков
- Евклидово расстояние между эмбедингами CNN

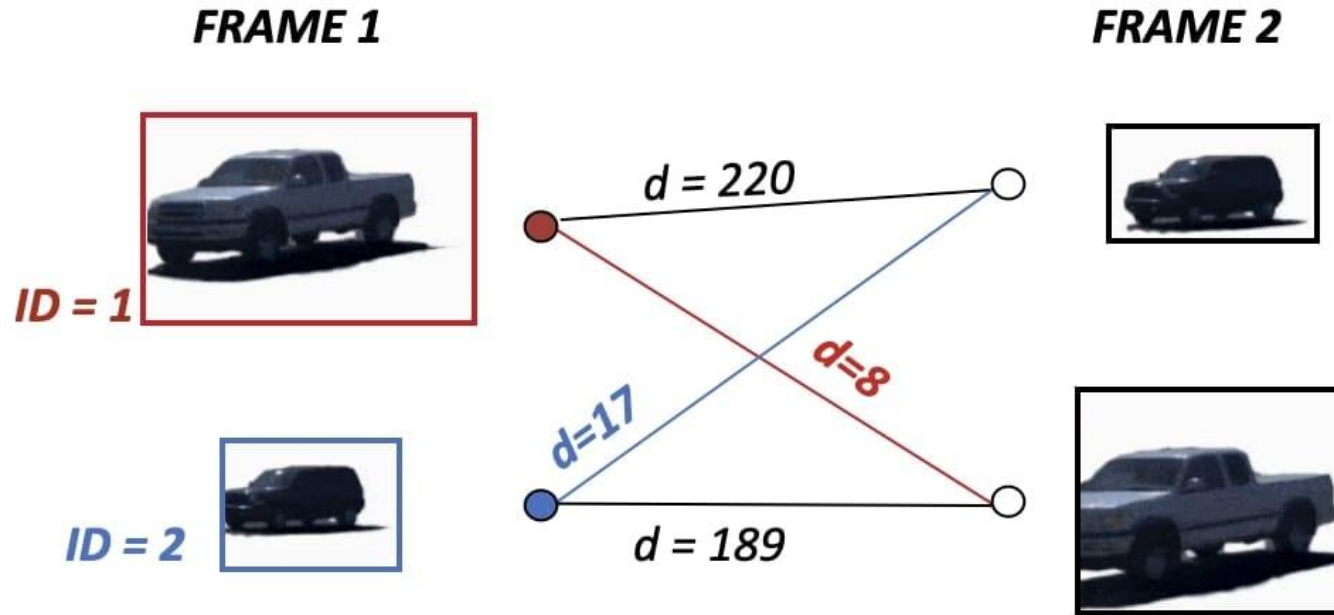
АССОЦИАЦИЯ ДАННЫХ: ДВУДОЛЬНОЕ СООТВЕТСТВИЕ

Рассмотрим пример: если в качестве метрики сходства выбрать **евклидово расстояние**, можно вычислить расстояние между центрами объектов. В данном случае расстояние между красной точкой (исходным объектом) и левой машиной минимально, тогда как до правой машины — значительно больше. Такое сопоставление позволяет определить наиболее вероятное соответствие.



АССОЦИАЦИЯ ДАННЫХ: ДВУДОЛЬНОЕ СООТВЕТСТВИЕ

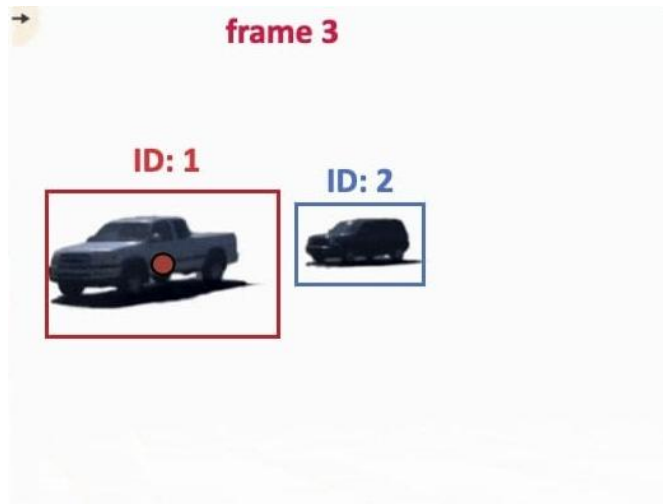
Далее формируется **двудольный граф**, в котором хранятся все вычисленные расстояния между объектами двух последовательных кадров.



АССОЦИАЦИЯ ДАННЫХ: ДВУДОЛЬНОЕ СООТВЕТСТВИЕ

На заключительном этапе ассоциации каждому объекту во втором кадре (t) присваивается идентификатор, соответствующий объекту из кадра ($t-1$).

Сопоставление выполняется по принципу **минимального расстояния** — объекту назначается ID того элемента, с которым зафиксировано наименьшее значение метрики сходства.



3. ПРОГНОЗИРОВАНИЕ ПОЛОЖЕНИЯ: ФИЛЬТР КАЛМАНА

Для повышения устойчивости отслеживания и компенсации временных пропусков детектора используется **фильтр Калмана (Kalman Filter)**. Он представляет собой рекурсивный алгоритм оценки состояния динамической системы, основанный на модели движения и наблюдениях, полученных от детектора.

Принцип работы

В контексте отслеживания объектов фильтр Калмана прогнозирует положение и скорость объекта в следующем кадре, используя данные о предыдущих состояниях.

Пусть **красная рамка** обозначает положение объекта на кадре $t-1$, а **оранжевая** — прогнозируемое положение на кадре t .

Вместо прямого сопоставления рамок между кадрами $t-1$ и t , мы сопоставляем рамку t с **прогнозом**.



ПРОГНОЗИРОВАНИЕ ПОЛОЖЕНИЯ: ФИЛЬТР КАЛМАНА

Двухэтапная работа фильтра

1

Прогноз (Prediction)

На основе модели движения (например, модель с постоянной скоростью) предсказывается следующее состояние объекта:

$$\hat{x}_{k|k-1} = F_k x_{k-1} + B_k u_k$$

где F_k — матрица перехода состояния,
 x_{k-1} — предыдущее состояние

2

Коррекция (Update)

Прогноз корректируется на основе фактических наблюдений (детекций):

$$x_k = \hat{x}_{k|k-1} + K_k (z_k - H_k \hat{x}_{k|k-1})$$

где K_k — коэффициент усиления Калмана,
 z_k — измерение

Постепенно алгоритм адаптируется и всё точнее предсказывает положение объектов во времени, обучаясь на наблюдаемой динамике каждого конкретного объекта.

Комбинация описанных методов (детекция + ассоциация данных + фильтр Калмана) реализует алгоритм **SORT (Simple Online and Real-time Tracking)** — один из наиболее известных и эффективных подходов для многокадрового отслеживания объектов в реальном времени, используемый в качестве базового решения во многих практических приложениях.

ПРОБЛЕМА СОПОСТАВЛЕНИЯ ОБЪЕКТОВ ПРИ ИЗМЕНЕНИИ ИХ КОЛИЧЕСТВА ВО ВРЕМЕНИ

Рассмотрим ситуацию, когда два объекта находятся на минимальном расстоянии от одного и того же объекта на предыдущем кадре. Это приводит к **неоднозначности при определении соответствий** между объектами на последовательных изображениях.

Например, в одной сцене один объект может покидать поле зрения, а другой — появляться. В таком случае объект 1 перемещается, а на следующем кадре появляется новый, третий объект.

Возникает вопрос: как корректно установить соответствие между объектами, если один из них сместился слишком далеко, и теперь несколько объектов претендуют на сопоставление с одним и тем же элементом предыдущего кадра?

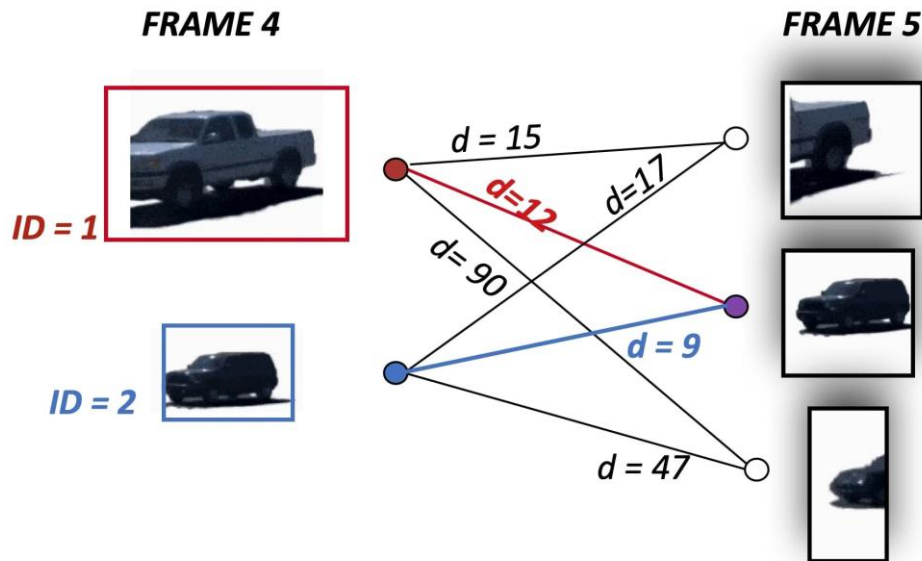


ПРОБЛЕМА СОПОСТАВЛЕНИЯ ОБЪЕКТОВ ПРИ ИЗМЕНЕНИИ ИХ КОЛИЧЕСТВА ВО ВРЕМЕНИ

Такую ситуацию можно формально представить в виде **двудольного графа**, где вершины одной доли соответствуют объектам первого кадра, а вершины другой доли — объектам второго кадра. Рёбра между ними задаются по критерию близости (например, евклидова расстояния), а веса рёбер отражают степень вероятности соответствия.

В ряде случаев один и тот же объект может быть связан сразу с двумя элементами другой доли, что делает задачу **неоднозначной**. Подобные ситуации возникают достаточно часто, особенно при анализе сцен, где количество объектов в последовательных кадрах различается.

Например, в одном кадре может быть три транспортных средства, а в следующем — пять; или же часть объектов может быть детектирована некорректно.



АССОЦИАЦИЯ ДАННЫХ: ВЕНГЕРСКИЙ АЛГОРИТМ

Для решения всех подобных задач — включая случаи перекрытий, появления и исчезновения объектов, а также ошибок детекции — используется **Венгерский алгоритм (Hungarian Algorithm)**. Этот алгоритм позволяет оптимально решить задачу минимизации суммарных стоимостей сопоставления между элементами двух множеств и лежит в основе большинства современных систем отслеживания и распознавания объектов, включая системы, использующие методы глубокого обучения.

Для иллюстрации принципа работы Венгерского алгоритма рассмотрим классический пример — **задачу оптимального распределения заданий между исполнителями**.

Пусть существует организация, сотрудники которой выезжают для выполнения полевых задач — например, для оценки объектов недвижимости. Каждый объект, подлежащий оценке, является заданием, а каждый сотрудник — исполнителем. Необходимо распределить сотрудников по заданиям таким образом, чтобы суммарные затраты — например, расстояния, которые сотрудники должны преодолеть до объектов, — были минимальны.

Ключевое ограничение

Каждое задание выполняется ровно одним сотрудником

АССОЦИАЦИЯ ДАННЫХ: ВЕНГЕРСКИЙ АЛГОРИТМ

Для формализации этой задачи построим **матрицу затрат** размером 3×3 , где строки соответствуют сотрудникам, а столбцы — заданиям.

Каждый элемент матрицы c_{ij} обозначает расстояние между i -м сотрудником и j -м объектом недвижимости.

Визуальный анализ матрицы может привести на мысль, что все сотрудники должны выбрать одно и то же задание, например, то, которое расположено ближе всего. Однако очевидно, что такой вариант невозможен — каждое задание должно быть выполнено ровно одним сотрудником.

	Job 1	Job 2	Job 3
Worker A	10	15	9
Worker B	9	18	5
Worker C	6	14	3

ВЕНГЕРСКИЙ АЛГОРИТМ

Пусть исходная матрица затрат G имеет размер $n \times n$ (в нашем примере $n=3$). Алгоритм выполняется в пять шагов.

01

Вычитание минимального значения по строкам

Из каждого элемента каждой строки вычитается минимальное значение в этой строке. Таким образом, в каждой строке появляется как минимум один нулевой элемент.

02

Вычитание минимального значения по столбцам

Аналогично первому шагу, из каждого элемента каждого столбца вычитается минимальное значение в данном столбце. После этого этапа в каждом столбце также будет как минимум один нулевой элемент.

03

Покрывие нулей минимальным числом линий

Необходимо провести минимальное количество горизонтальных и вертикальных линий, перекрывающих все нули в матрице. Если количество линий меньше n , то оптимальное решение ещё не достигнуто.

04

Коррекция матрицы

Если количество линий меньше n , находится минимальный элемент, не покрытый ни одной линией. Это значение вычитается из всех непокрытых элементов, а элементы на пересечении линий увеличиваются на это значение.

05

Назначение исполнителей

Когда количество линий становится равным n , можно перейти к построению окончательного распределения. Назначения выполняются, начиная со строк, содержащих ровно один ноль.

Результатом работы алгоритма является **оптимальное распределение**, при котором суммарные затраты минимальны.

РЕШЕНИЕ ЗАДАЧИ

Шаг 1. Вычитание минимального значения по строкам

1) Remove the minimum of each row

	Job 1	Job 2	Job 3
Worker A	10	15	9
Worker B	9	18	5
Worker C	6	14	3



	Job 1	Job 2	Job 3
Worker A	1	6	0
Worker B	4	13	0
Worker C	3	11	0

В исходной матрице слева мы видим, что нужно удалить числа 9, 5 и 3. Мы вычтем 9 из первой строки, 5 из второй строки и 3 из третьей строки.

РЕШЕНИЕ ЗАДАЧИ

Шаг 2. Вычитание минимального значения по столбцам

2) Remove the minimum of each column

	Job 1	Job 2	Job 3
Worker A	1	6	0
Worker B	4	13	0
Worker C	3	11	0



	Job 1	Job 2	Job 3
Worker A	0	0	0
Worker B	3	7	0
Worker C	2	5	0

Вычитаем 1 из первого столбца, 6 из второго столбца и 0 из третьего столбца

РЕШЕНИЕ ЗАДАЧИ

Шаг 3. Покрытие нулей минимальным числом линий

3) Cross 0s with the minimum number of lines possible

	Job 1	Job 2	Job 3
Worker A	0	0	0
Worker B	3	7	0
Worker C	2	5	0



	Job 1	Job 2	Job 3
Worker A	0	0	0
Worker B	3	7	0
Worker C	2	5	0



На рисунках левый переход неправильный, потому что для покрытия всех нулей требуется 5 линий, в то время как можно сделать это с помощью всего 2 линий, как в примере справа.

РЕШЕНИЕ ЗАДАЧИ

Шаг 4. Коррекция матрицы

4) Subtract the smallest element of the matrix. If a number has been crossed twice, add the element to it.

	Job 1	Job 2	Job 3
Worker A	0	0	0
Worker B	3	7	0
Worker C	2	5	0



	Job 1	Job 2	Job 3
Worker A	0	0	2
Worker B	1	5	0
Worker C	0	3	0

2 — это наименьший элемент, поэтому мы вычитаем его из всех чисел. Таким образом, в матрице появляются новые нули, и процесс возвращается к шагу 3.

РЕШЕНИЕ ЗАДАЧИ

Шаг 3. Покрытие нулей минимальным числом линий

Cross the 0s again and check the number of lines

	Job 1	Job 2	Job 3
Worker A	0	0	2
Worker B	1	5	0
Worker C	0	3	0



	Job 1	Job 2	Job 3
Worker A	0	0	2
Worker B	1	5	0
Worker C	0	3	0

Нам нужно 3 строки, чтобы пересечь нули, и в этот раз $3 = n$

РЕШЕНИЕ ЗАДАЧИ

Шаг 5. Назначение исполнителей

	Job 1	Job 2	Job 3
Worker A	0	0	2
Worker B	1	5	0
Worker C	0	3	0

Назначения выполняются, начиная со строк, содержащих ровно один ноль. После сопоставления строки и столбца, соответствующие этому нулю, исключаются из дальнейшего рассмотрения. Начнём со второй строки (там только один ноль) и соединим задание 3 с работником В, а затем пересечём линии и продолжим.

РЕШЕНИЕ ЗАДАЧИ

Результатом работы алгоритма является оптимальное распределение, при котором суммарные затраты минимальны.

	Job 1	Job 2	Job 3
Worker A	10	15	9
Worker B	9	18	5
Worker C	6	14	3



Job 1	Worker C
Job 2	Worker A
Job 3	Worker B

Финальный ответ

ВОЗМОЖНЫЕ ФУНКЦИИ ЗАТРАТ

При построении матрицы затрат необходимо выбрать метрику, которая будет определять степень "схожести" или "расстояния" между рамками в двух кадрах. Вспомним геометрические критерии сопоставления (слайд № 13)

1

Евклидово расстояние

Используется для сопоставления объектов на основе близости их центров:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Этот подход прост, но не учитывает изменения формы или перекрытия объектов.

2

Пересечение по объединению (IoU)

В этом случае стоимость определяется через метрику Intersection over Union (IoU). Такой метод подходит для задач, где важно учитывать степень перекрытия объектов. Следует отметить, что использование IoU меняет характер задачи: в отличие от минимизации расстояния, здесь необходимо **максимизировать** показатель перекрытия.

3

Свёрточная стоимость (визуальное сходство)

В случаях, когда объекты имеют схожие размеры и могут частично перекрываться, полезно учитывать визуальное сходство содержимого рамок. Для этого применяется **свёрточная нейронная**, вычисляющая **косинусное расстояние** между признаковыми векторами изображений.

ПОЛЬЗОВАТЕЛЬСКАЯ ФУНКЦИЯ ЗАТРАТ

В реальных задачах возможно комбинирование нескольких метрик для формирования **обобщённой функции стоимости**.

Например, общая стоимость может учитывать одновременно:

- пространственное расстояние между центрами рамок;
- степень перекрытия (IoU);
- визуальное сходство признаков;
- различие в размерах или форме рамок;
- принадлежность к определённому классу объектов.

Для решения любой задачи можно разработать собственную функцию стоимости, преобразовать её в двудольный граф, а затем применить к задаче венгерский алгоритм.

Таким образом, итоговая функция затрат может быть записана как:

$$C_{ij} = \alpha \cdot d_{\text{евк}} + \beta \cdot (1 - \text{IoU}) + \gamma \cdot d_{\text{виз}},$$

где коэффициенты α, β, γ задают относительные веса различных факторов.

ПРЯМОУГОЛЬНАЯ МАТРИЦА (N×M)

Венгерский алгоритм требует **квадратной** матрицы. Если $N \neq M$:

Шаг А. Добавить фиктивные строки или столбцы, чтобы получить $n = \max(N, M)$.

Шаг В. Заполнить добавленные ячейки **стоимостью «нет назначения»** τ .

- В задачах **минимизации** (например, с матрицей стоимостей) τ выбирают как разумный штраф за «рождение/смерть трека» — достаточно большой, чтобы алгоритм предпочитал реальные соответствия, но не бесконечный.
- В задачах **максимизации** (до преобразования к минимизации) τ задают после преобразования.

Важно: добавлять максимальное значение из матрицы в новый столбец — неуниверсально. Правильнее ввести осмысленный штраф τ , отражающий цену появления/исчезновения объекта, а не механически дублировать максимум.

	Tracking 1	Tracking 2	Tracking 3		Tracking 1	Tracking 2	Tracking 3	Tracking 4
Detection 1	21	10	0		21	10	0	90
Detection 2	56	62	11		56	62	11	90
Detection 3	20	77	90		20	77	90	90
Detection 4	10	80	20		10	80	20	90



ОТ МАКСИМИЗАЦИИ К МИНИМИЗАЦИИ

Венгерский алгоритм решает **минимизацию**. Если изначально решается задача **максимизации** (например, IoU, cosine-similarity), то необходимо преобразовать матрицу сходства $S=(s_{ij})$ в матрицу **стоимостей** $C=(c_{ij})$.

Три эквивалентных и корректных способа:

1.Линейная инверсия к единице (часто в трекинге с IoU): $c_{ij} = 1 - \text{IoU}_{ij}$ ($\text{IoU} \in [0, 1]$).

2.Негация с сдвигом: $c_{ij} = s_{\max} - s_{ij}$, $s_{\max} = \max_{i,j} s_{ij}$.

3.Негация (если реализация допускает отрицательные стоимости): $c_{ij} = -s_{ij}$.

Способ (1) удобен и интерпретируем: «чем больше перекрытие, тем меньше стоимость». После преобразования задается штраф τ в той же шкале, что и c_{ij} . Для IoU это обычно $\tau \in [0.4, 0.7]$ (т.е. эквивалент IoU-порога 0.3-0.6).

	Tracking 1	Tracking 2	Tracking 3	Tracking 4
Detection 1	21	10	0	90
Detection 2	56	62	11	90
Detection 3	20	77	90	90
Detection 4	10	80	20	90



	Tracking 1	Tracking 2	Tracking 3	Tracking 4
Detection 1	69	80	90	0
Detection 2	34	28	79	0
Detection 3	70	13	0	0
Detection 4	80	10	70	0

$80 = 90 - 10$



РАСШИРЕННЫЕ МОДЕЛИ ОТСЛЕЖИВАНИЯ НЕСКОЛЬКИХ ОБЪЕКТОВ

В ранее рассмотренном примере мы выполняли два шага:

1.Обнаружение автомобилей на каждом кадре видеопоследовательности.

2.Отслеживание их движения между кадрами.

Такой подход называется **отслеживанием на основе обнаружения (Tracking-by-Detection)**.

Он предполагает, что процесс детектирования объектов и процесс их ассоциации (сопоставления между кадрами) выполняются **раздельно**.

Однако современные методы позволяют выполнять **совместное обнаружение и отслеживание**, при котором оба процесса интегрированы в одну модель.

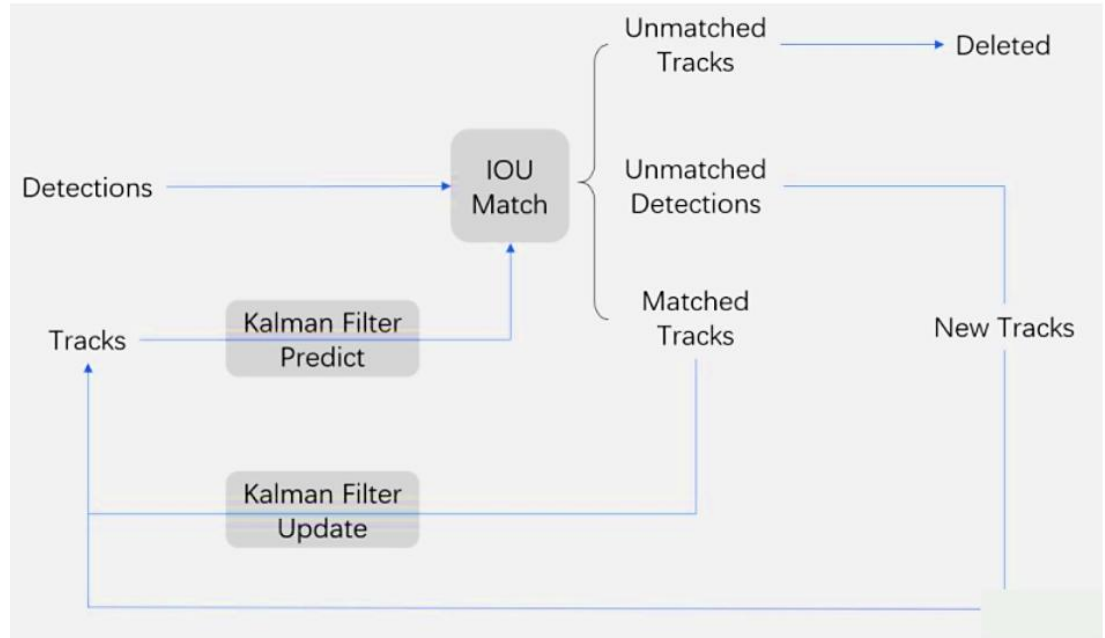
Таким образом, выделяют два основных класса подходов:

Отслеживание на основе алгоритмов обнаружения (Tracking-by-Detection).

Совместные алгоритмы обнаружения и отслеживания (Joint Detection and Tracking).

ОТСЛЕЖИВАНИЕ НА ОСНОВЕ ОБНАРУЖЕНИЯ

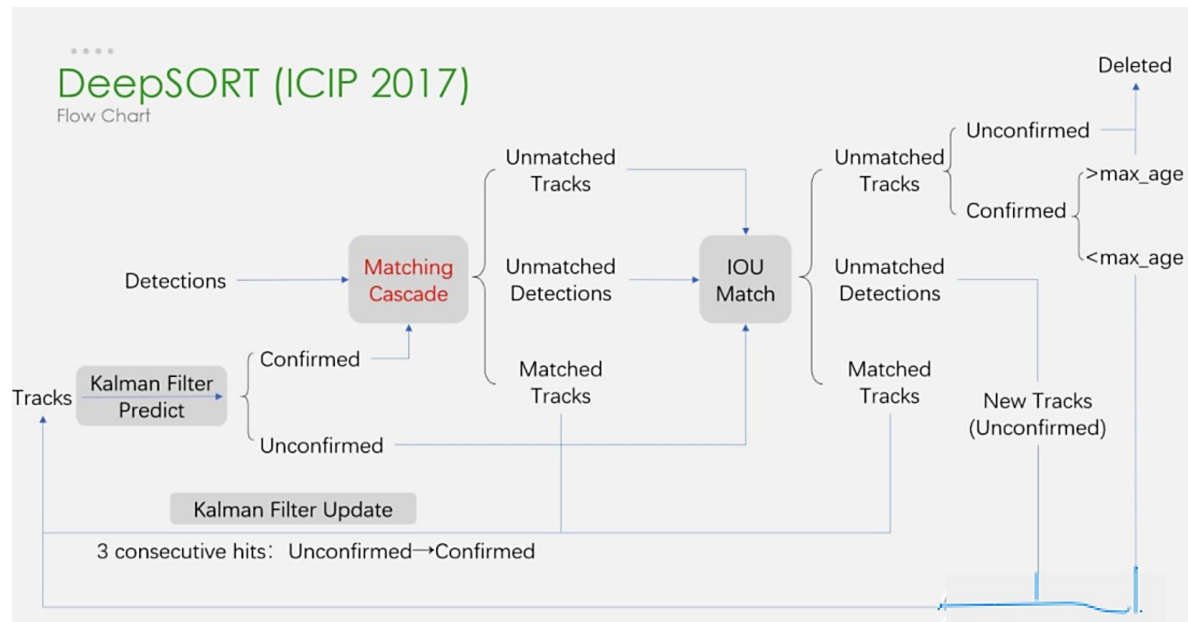
Ранее мы уже познакомились с базовым примером такого подхода — алгоритмом **SORT (Simple Online and Realtime Tracking)**. SORT относится к семейству методов, в которых трекер получает на вход результаты детектора (например, YOLO или Faster R-CNN) и связывает их между последовательными кадрами с помощью **фильтра Калмана** и алгоритма оптимального сопоставления (Венгерского алгоритма).



DEEP SORT

Если к такому подходу добавить **глубокие признаки**, извлекаемые с помощью свёрточных нейронных сетей, то получается более продвинутый метод — **Deep SORT**.

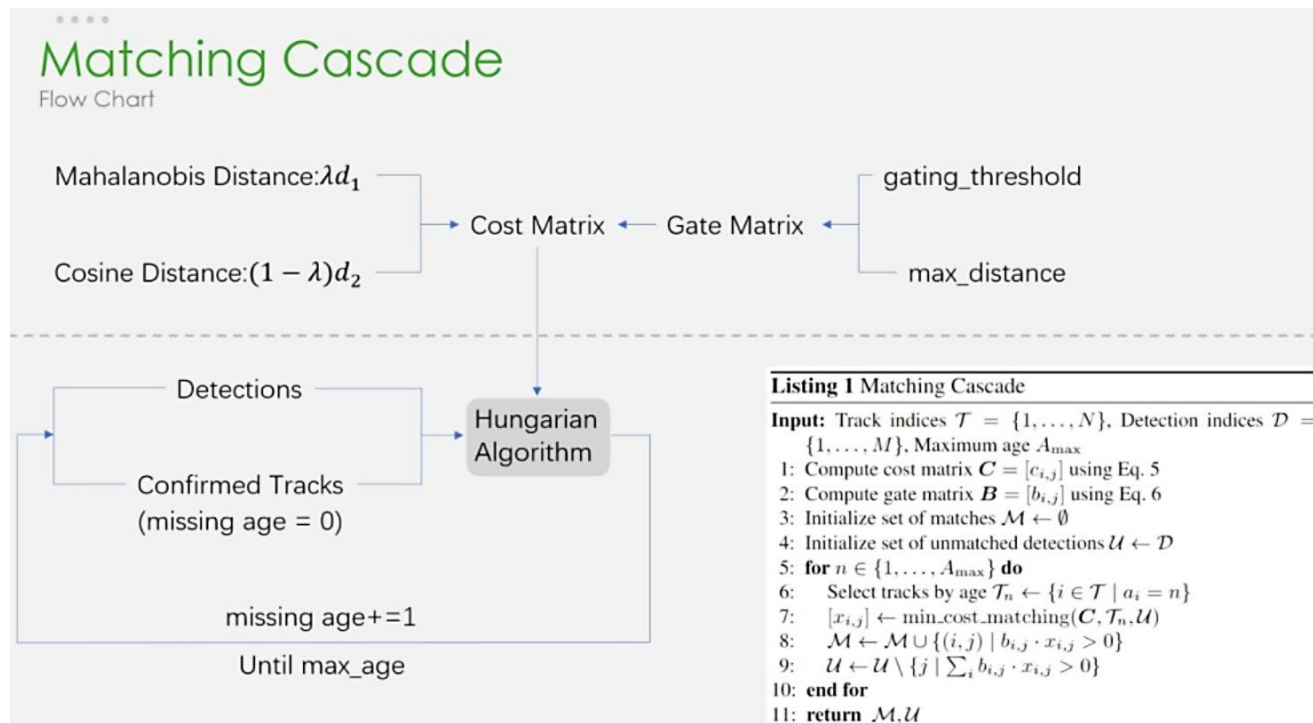
Идея Deep SORT заключается в том, что при сопоставлении областей детекции используется не только геометрическое расстояние, но и сходство между признаковыми представлениями (feature embeddings), полученными из нейронной сети. Таким образом, сопоставление учитывает не только форму и положение объектов, но и их внутренние визуальные характеристики.



DEEP SORT

Блок-схема каскадного сопоставления выглядит следующим образом:

Верхняя часть — это оценка сходства, которая является функцией стоимости в этой задаче распределения. В нижней части по-прежнему используется венгерский алгоритм для сопоставления кадра обнаружения и кадра отслеживания.

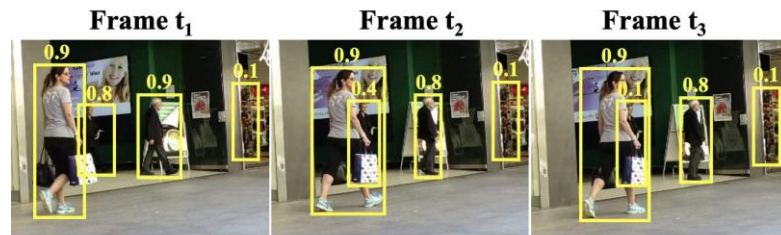


BYTETRACK

Основной принцип ByteTrack заключается в применении двухэтапной ассоциации детекций:

1. На первом этапе сопоставляются детекции с **высокой степенью достоверности** (high-confidence detections).
2. На втором этапе оставшиеся низкоуверенные детекции (low-confidence) могут быть сопоставлены с уже существующими треками, если это согласуется с их предсказанными траекториями.

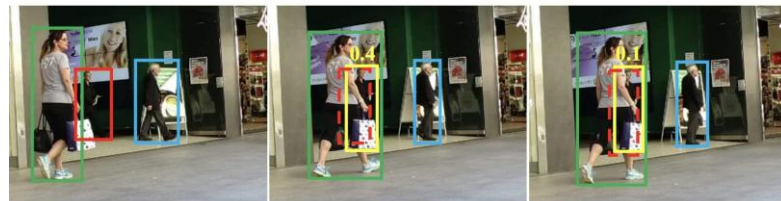
Такой подход особенно полезен в ситуациях, когда уверенность детектора непостоянна или внешние условия (например, дождь, освещение, ракурс) затрудняют точное распознавание объектов. В результате ByteTrack способен восстанавливать треки даже для объектов, временно потерянных из-за низкой достоверности детекции.



(a) detection boxes



(b) tracklets by associating high score detection boxes



(c) tracklets by associating every detection box

СОВМЕСТНОЕ ОБНАРУЖЕНИЕ И ОТСЛЕЖИВАНИЕ

Следующее семейство методов — это совместные алгоритмы обнаружения и отслеживания, которые решают обе задачи (детекцию и трекинг) в рамках **одной** нейросетевой архитектуры. В отличие от классического подхода “детектор + трекер”, здесь сеть одновременно обучается определять объекты и поддерживать их идентичности между кадрами.

- ❑ Одним из наиболее известных представителей является **FairMOT (Fair Multi-Object Tracking)**.

Это система, специально разработанная для решения задачи **совместного обнаружения и повторной идентификации (Re-Identification, ReID)**.

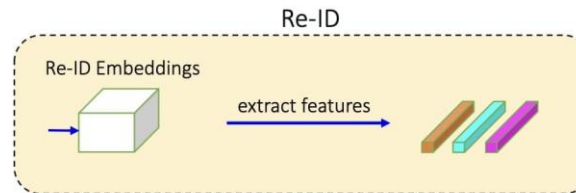
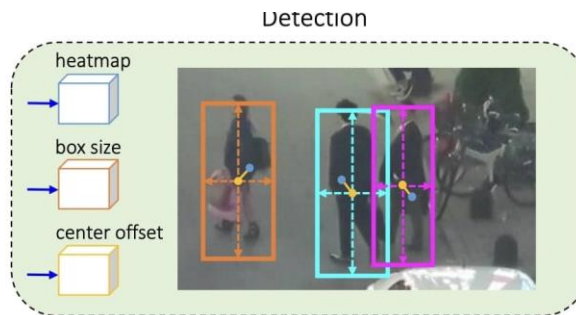
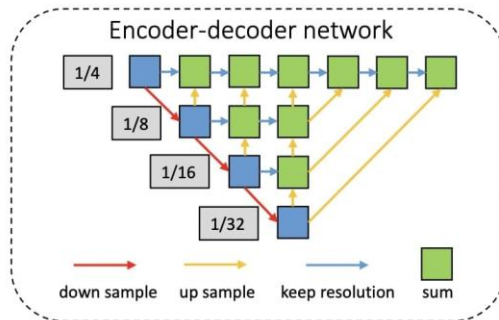
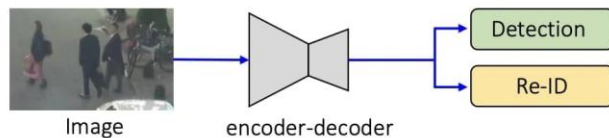
FAIRMOT

FairMOT имеет **две параллельные ветви**:

1. Первая ветвь отвечает за **обнаружение объектов** (используется безякорная архитектура, *anchor-free*).
2. Вторая ветвь занимается **извлечением признаков для ReID**, что позволяет различать объекты с похожими внешними характеристиками.

Такой подход обеспечивает баланс между скоростью работы и точностью отслеживания.

FairMOT показывает высокую эффективность в сложных сценах: при наличии препятствий, плотной толпы или частичных перекрытий.



КРАТКИЕ СВЕДЕНИЯ И ВЫВОДЫ

DETECTION



YOLO, ...

- Bounding Box coordinates
- Class

ASSOCIATION



+ Hungarian Algorithm

- Bounding Box coordinates
- Class
- ID
- Age

TRACKING



+ Kalman Filters

- Bounding Box coordinates
- Class
- ID
- Age
- Uncertainty
- Motion Vectors

PREDICTION



+ Forecasting

- Bounding Box coordinates
- Class
- ID
- Age
- Uncertainty
- Motion Vectors
- Future Positions
- Action/Intent
- Future Trajectory



Управление данными и искусственный интеллект
Лекция по дисциплине:
“Глубокое обучение в компьютерном зрении”



Российский университет
дружбы народов

ВИЗУАЛЬНОЕ ОТСЛЕЖИВАНИЕ ДВИЖУЩИХСЯ ОБЪЕКТОВ

Преподаватели: Перчихин Олег Игоревич
Федоренко Анастасия Олеговна